DOSSIER MAÎTRE · PLAN DE DÉVELOPPEMENT

CHEZVOUS 2.0

Architecture, produit, réservation, paiements et stratégie GitHub

Plan opérationnel détaillé pour bâtir une plateforme de séjours fiable et économique au Sénégal, préparée pour la location longue durée, la vente et l'IA.

79 TÂCHES · 8 LOTS · 3 ESPACES

Version 1.0 — 18 octobre 2026 · Document de cadrage, non implémentation terminée

LECTURE ET UTILISATION

Sommaire

00 Résumé exécutif

01 Produit et fonctionnalités

02 Architecture technique

03 Schéma de données

04 API et paiements

05 Sécurité et conformité

06 Développement par lots

07 Références GitHub

08 Tests et exploitation

09 Business model et KPI

> Ce dossier présente des architectures et tarifs **proposés**. Les commissions, le PSP, les obligations légales et les budgets nécessitent validation avant lancement. Aucun dépôt GitHub, serveur ou base de données du projet n'a été modifié lors de cette préparation.

CHEZVOUS — Dossier maître de développement 2026

Version : 1.0 — 18 octobre 2026
Statut : proposition d'architecture et de planification ; **aucun développement réel n'est présumé achevé**.
Marché initial : Sénégal (Dakar), séjours en appartements meublés, villas, maisons et unités de résidences.
Modèle : marketplace B2B2C + outil de gestion pour hôtes/agences.

Décision produit

Développer **CHEZVOUS Séjours + CHEZVOUS Host + CHEZVOUS Control** dans une seule application web mobile-first/PWA. Acquérir d'abord 20 à 30 unités autorisées et contrôlées auprès d'un nombre restreint de professionnels. Permettre la réservation sur demande, le paiement sécurisé par un PSP approprié et l'administration complète de chaque dossier. Les réservations instantanées sont réservées à un inventaire répondant à des critères de synchronisation et de fiabilité.

Livrables du dossier

1. `docs/01_PRODUIT.md` : objectifs, personas, fonctionnalités, parcours et périmètre.
2. `docs/02_ARCHITECTURE.md` : stratégie technique, modules, flux, intégrations et invariants.
3. `docs/03_DONNEES.md` : modèle conceptuel, schéma des entités, états et contraintes.
4. `docs/04_API_ET_PAIEMENTS.md` : contrats API, moteur de réservation, paiements, iCal.
5. `docs/05_SECURITE_CONFORMITE.md` : RLS, risques, obligations et exigences.
6. `docs/06_ROADMAP.md` : lots, responsabilités, critères de validation et budget.
7. `docs/07_GITHUB_OPEN_SOURCE.md` : dépôts audités, licences, critères de réutilisation.
8. `docs/08_TESTS_OPERATIONS.md` : stratégie qualité, observabilité, exploitation.
9. `docs/09_BUSINESS_KPIS.md` : modèle économique, mesures et simulations.
10. `backlog_chezvous.csv` : backlog importable dans GitHub Projects.
11. `architecture/chezvous-erd.mmd` et `architecture/chezvous-flux.mmd` : diagrammes Mermaid.
12. `.github/workflows/ci.example.yml` : exemple de contrôle continu, non prêt à être activé sans code.

Grands principes non négociables

- La réservation doit être **atomique**, sans double occupation possible dans la base métier.
- Les soldes financiers sont issus de faits de paiement vérifiés par serveur, jamais d'un retour navigateur.
- Une annonce n'est publiée qu'après les contrôles nécessaires à sa catégorie.
- Les prix sont présentés en FCFA/XOF avec devis final figé à la réservation.

- Les informations privées (identité, documents, adresse exacte, téléphones) sont protégées.
- L'interface doit conserver l'identité graphique ChezVous préalablement validée ; ne pas inventer de logo alternatif.
- Modules futurs préparés par le schéma, mais non imposés à la V1 : bail, vente, CRM avancé, IA, conciergerie.
- Réutilisation GitHub uniquement après audit de licence, activité, risques et tests.

État d'avancement vérifiable

Phase	Statut
Analyse Sénégal + internationale	Recherche documentaire réalisée ; audit transactionnel non réalisé
Cadrage MVP et architecture cible	Proposition détaillée dans ce dossier
Développement, tests, production	Non vérifiés ; ne pas inférer un taux de progression
Recrutement des premiers logements	Non confirmé

Décisions bloquantes avant démarrage

- Confirmer la marque et les fichiers source de la charte validée.
- Contrôler les catégories et obligations juridiques des partenaires d'hébergement.
- Choisir le prestataire de paiement, les frais, la politique de remboursement et les modalités de reversement.
- Confirmer le scénario principal de réservation (demande avant paiement ou autorisation/ prépaiement via prestataire).
- Désigner les rôles opérationnels et le propriétaire du projet GitHub, Supabase et Vercel.

01 — Vision produit, périmètre et expérience

1. Proposition de valeur

ChezVous est **une plateforme de séjours réservables et vérifiés**, combinant une marketplace destinée au voyageur et un système de gestion professionnel pour les propriétaires, résidences et agences. Le bien n'est pas seulement affiché : il dispose d'un exploitant responsable, d'une unité réservable, d'un calendrier, de règles de prix, de conditions contractuelles, d'un suivi de paiement et d'un dossier de séjour.

Positionnement économique

- Voyageur : rechercher librement, voir le prix final, réserver en confiance et obtenir une assistance.
- Hôte/gestionnaire : publier simplement, recevoir des réservations, gérer ses disponibilités et diffuser son propre lien de réservation.
- ChezVous : percevoir la commission de distribution lorsqu'un séjour provient de la marketplace ; proposer des abonnements professionnels et des services à la carte.
- Hypothèse de prix de lancement : 7 % de commission sur réservations marketplace ; abonnements professionnels à tester autour de 19 900 et 49 900 FCFA/mois ; aucun tarif n'est validé commercialement.

2. Segments du pilote

1. Appartements meublés professionnels à Dakar : clientèle affaires, visiteurs, diaspora et familles.
2. Villas et maisons de vacances : activées après validation opérationnelle, particulièrement Saly et Somone.
3. Résidences : propriété physique unique pouvant contenir plusieurs unités réservable séparément.

Exclus du lancement commercial : terrains, ventes de biens, location longue durée et financement immobilier. Leur futur modèle de données sera néanmoins réservé.

3. Personas et objectifs

Profil	Besoin essentiel	Action déterminante
Voyageur local	Prix transparent et disponibilité réelle	Confirmer un séjour sans conflit
Voyageur diaspora	Confiance à distance, paiement clair	Réserver et recevoir les consignes
Hôte indépendant	Gestion simple sans outil complexe	Publier et gérer calendrier/réservations
Gestionnaire de résidence	Plusieurs appartements et collaborateurs	Contrôler portefeuille et disponibilités
Agent de vérification	Vérifier identité, mandat et conformité	Rendre une unité publiable ou la bloquer
Support/finance	Gérer litiges, paiements et remboursements	Reconstituer chaque événement et décision

4. Fonctionnalités MVP — Voyageur

- Accueil mobile-first et recherche par destination, dates, voyageurs et fourchette de prix.
- Liste et carte simple de résultats ; état de disponibilité contrôlé avant confirmation.
- Filtres : type, chambres, capacité, climatisation, cuisine, internet, parking, piscine (si attestés).
- Détail : photos optimisées, contenu vérifiable, quartier approximatif, règles du logement, équipements, prix ventilé, annulation.
- Compte après la phase exploratoire, connexion e-mail ou téléphone selon coût et fiabilité du fournisseur.
- Devis figé : montant hébergement, frais de service, taxes éventuelles, total XOF.
- Réservation sur demande, délai d'acceptation connu, état clair (en attente, confirmée, refusée, etc.).
- Paiement via le prestataire agréé/autorisé choisi et facture/reçu adaptés au montage contractuel.
- Espace Mes séjours ; messages système ; assistance et demande d'annulation.
- Gestion de l'avis **seulement après un séjour honoré** ; facultatif en version 1.1.

5. Fonctionnalités MVP — Hôte / Agence

- Onboarding : type d'acteur, organisation, coordonnées, identité et justificatifs selon les obligations.
- Fiche bâtiment / propriété puis unités réservables ; adresses et coordonnées séparées des informations publiques.
- Disponibilités quotidiennes, règles de durée minimale, blocages, tarifs et exceptions de prix.
- Gestion des demandes : accepter/refuser dans le délai, sans pouvoir supprimer une réservation confirmée sans procédure.
- Suivi des séjours, statuts de paiements, rapprochement financier (données PSP), exports simples.
- Mini-page publique et lien de réservation directe à envisager dès MVP+, commercialisation avec Pro en V1.5.
- Gestion multi-utilisateurs avec rôles propriétaire, gestionnaire et personnel ; invitations journalisées.

6. Fonctionnalités MVP — Administrateur

- File de validation des propriétaires, mandataires, documents et unités ; procédures de contrôle.
- Dossiers de réservation : calendrier, journal d'événements, paiement, incidents, annulation et résolution.
- Gestion des signalements, gel des annonces, suspension des comptes, correction contrôlée.
- Vue des paiements : état PSP, références externes, remboursements initiés/confirmés ; aucune modification manuelle d'un paiement réussi sans événement vérifiable.
- Droits d'accès élevés restreints, journal audit immuable au niveau applicatif, approbation des exceptions.
- Tableaux de bord : unités publiées, taux de réponses, conversions, annulations, incidentologie, revenu brut, commissions et contributions.

7. Parcours critiques

Voyageur : rechercher → ouvrir la fiche → vérifier le prix final → choisir les dates → créer une demande → hôte accepte / délai expire → paiement via PSP → confirmation double côté serveur → check-in → check-out → avis.

Hôte : inscription → création organisation → contrôle documentaire → propriété/unité → calendrier et tarifs → inspection/modération → publication → notification de demande → acceptation → gestion du séjour.

Administrateur : KYC/mandat → publication → surveillance et assistance → revue incident/annulation → rapprochement paiement → indicateurs qualité.

8. Règles produit fondamentales

- Une propriété peut contenir plusieurs unités : une villa entière, un appartement, plusieurs appartements d'une résidence.
- L'unité peut être vendue ou louée plus tard, sans recréation ; une offre **stay** est distincte d'une offre future **lease** et **sale**.
- Une même unité ne peut avoir qu'une réservation active bloquante pour les mêmes nuits ; date de sortie exclusive.
- Les prix et conditions acceptés sont figés sur le devis/contrat ; les nouvelles règles ne changent pas rétroactivement une réservation.
- Aucune disponibilité ne peut être garantie si les calendriers externes ne sont pas fiables : gestion des unités en mode demande et contrôles renforcés.
- La facture et la reconnaissance de chiffre d'affaires sont des sujets comptables/juridiques distincts de l'écran de paiement.

9. Qualité de l'interface

- Direction : premium, simple, très moderne, palette ChezVous déjà validée et mode Light prioritaire.
- Responsive : 360 px → desktop ; interactions tactiles de 44 px minimum ; formulaires accessibles au clavier.
- Performance : viser LCP < 2,5 s sur les pages essentielles en conditions mobiles de référence ; images compressées et cache maîtrisé.
- Français en premier, anglais prévu ; wolof en couche de contenus localisables, pas de traductions codées en dur.
- Le design n'affiche jamais de données de disponibilité, de prix ou d'avis fictifs comme si elles étaient réelles.

10. Hors périmètre du MVP

Chat IA avancé, reconnaissance automatique de titres fonciers, visite 3D, home staging IA, applications iOS/Android natives, comptabilité agence complète, garanties locatives, crédit, escrow interne, maintenance sur site internalisée.

02 — Architecture logicielle et intégrations

1. Architecture de départ recommandée

Monolithe modulaire, TypeScript end-to-end : Next.js (App Router), React, Tailwind CSS, shadcn/ui, PostgreSQL/Supabase, Supabase Auth, Storage, Vercel, système d'e-mails transactionnels et prestataire de paiement habilité au Sénégal. Les appels tiers sont encapsulés derrière des interfaces adaptateurs.

Pourquoi ne pas commencer par des microservices ?

Le coût d'orchestration, les défaillances réseau, la duplication des logs et les transactions distribuées sont inutiles sur le premier catalogue. Des limites de modules propres, une base solide et des événements métiers suffisent. Les services peuvent être extraits si des métriques concrètes l'exigent.

2. Modules de domaine

Module	Responsabilités	Contrat stable
`identity`	Profil, sécurité, utilisateurs, invitations	user_id, membership
`organizations`	Sociétés, équipes, propriétaires, mandataires	org_id, memberships
`catalog`	Propriété, unité, équipements, médias, localisation	property_id / unit_id
`compliance`	Documents, contrôles, autorisations	review_id / policy_version
`pricing`	Calendrier des tarifs, règles et devis figés	quote_id / amount_xof
`availability`	Bloquages internes, réservation de nuits, iCal	unité + date-range
`booking`	Demande, acceptation, réservation, annulations	booking_id / state
`payments`	PSP, webhooks, remboursements, reversements	payment_id / provider_ref
`messaging`	Emails, notifications, modèles, journaux	notification_id
`operations`	Check-in, incident, pièces jointes, litiges	case_id
`analytics`	KPIs et exports normalisés	events normalisés
`offers`	Abstraction future : stay / lease / sale	offer_type

Dépendance descendante : **l'interface ne modifie jamais directement les tables critiques booking/payments** ; elle appelle des services serveur qui contrôlent le contexte utilisateur et réalisent les transactions SQL nécessaires.

3. Arborescence de repository proposée

```
chezvous/
  apps/
    web/
      src/app/(public)/          # accueil, recherche, logements
      src/app/(auth)/            # inscription/connexion
      src/app/(traveler)/        # séjours et profil
      src/app/(host)/            # unités, tarifs, demandes
      src/app/(control)/         # conformité, support, finance
      src/app/api/              # endpoints et callbacks
      src/components/           # design system ChezVous
  packages/
    domain/                      # entités, règles, transitions
    application/                 # use cases et services
    contracts/                   # schémas et DTO validés
    ui/                          # composants communs
    integrations/                # paiement, mail, iCal, carte
    config/                      # tsconfig/eslint/outils
  supabase/
    migrations/                  # SQL versionné
    tests/                       # permissions RLS et invariants
    seed.sql                     # données fictives non personnelles
  tests/
    e2e/                         # Playwright
    integration/                 # paiements, calendrier, RLS
  docs/
  .github/
    workflows/
    ISSUE_TEMPLATE/
```

La structure en `apps/` et `packages/` est recommandée si l'équipe prévoit plusieurs surfaces et libraries communes ; un dépôt unique avec `src/modules/` peut être encore plus économique pour 1-2 développeurs. Éviter l'abstraction inutile.

4. Séparation des données et stockage

- Données publiques : noms commerciaux, quartier, équipements, photos autorisées, prix et disponibilité non sensible.
- Données protégées : adresse exacte, documents de propriété, identités, coordonnées, historiques de paiements, conversations, motifs de litige.
- Buckets `public-listings` pour les images approuvées ; `private-documents` pour les justificatifs avec accès signé temporaire, RLS et journaux.
- Ne pas stocker de carte bancaire ni de secrets PSP. Chiffrer et limiter les données identifiantes, définir durée de conservation par usage.

5. Service de paiement interchangeable

Définir `PaymentGateway` avec `createPaymentIntent`, `getPaymentStatus`, `parseAndVerifyWebhook`, `refundPayment`, `payout` (si contractuellement disponible). Fournisseur A/B derrière adaptateur ; ne pas s'engager sur une fonctionnalité de split/remboursement non confirmée auprès du prestataire.

Paiement initié et confirmé uniquement côté serveur ; callbacks authentifiés et idempotents ; états financiers créés par événements vérifiés, rapprochement quotidien au minimum.

6. Disponibilité multicanal

V1 : disponibilité gérée dans ChezVous + blocages manuels ; demande de réservation pour tout calendrier externe non synchronisé.

V1.5 : import/export iCal lorsque compatible, synchronisation planifiée, horodatage dernière synchronisation, journal d'échec et alertes. iCal **n'est pas temps réel** : garder confirmation sur demande ou sécurisation renforcée. API Booking/Airbnb uniquement après accès contractuel et vérification des limitations fournisseurs.

7. Cartographie et recherche

PostgreSQL `PostGIS` plus tard si besoin ; pour le MVP, une latitude/longitude approximative publiée, le quartier et le prix suffisent. MapLibre est une bibliothèque de rendu, pas un fournisseur gratuit de tuiles, de géocodage ou de directions. Prévoir plafonds, caching et coût du fournisseur.

Recherche : SQL/Postgres FTS + filtres simples d'abord ; moteur de recherche spécialisé uniquement après mesure de lenteur. IA V2/V3 via interface `SearchAssistant`, fondée sur des résultats structurés et permissionnés.

8. Observabilité et opérations

- Logs structurés corrélés par `request_id`, `booking_id`, `payment_id` sans exposer de secrets/PII.
- Erreurs frontend et backend : traces et alertes (outil à choisir).
- Job outbox : enregistrement d'événement métier dans la même transaction que l'état, puis traitement asynchrone avec retry.
- Tâches planifiées : expiration de devis/holds, synchronisation iCal, réconciliation paiements, rappels, purge des données à échéance.
- Sauvegardes documentées, restauration testée, sauvegarde du stockage de documents si nécessaire (non couverte par tous les backups de base).

9. Environnements et livraison

Local, Preview/Staging, Production séparés ; jamais les identifiants de production en Preview. Migrer par Pull Request ; séparer les clés publiques des secrets, valider les migrations et exécuter tests automatiques avant déploiement. Infrastructure versionnée autant que possible.

10. Préparations dès V1 aux phases suivantes

- `offer_type` pour stay/lease/sale mais seules les offres stay sont exposées.
- `organization`, `ownership`, `mandate` et `document_scope` extensibles à bail et vente.
- Moteur de commission fondé sur la **source d'acquisition** et la grille contractuelle versionnée.
- Monnaie `currency_code`, montants entiers en XOF ; prise en charge future d'autres devises au niveau de la présentation seulement après décision commerciale.
- Langues clés de traduction, politiques et termes versionnés.

- API documentée par contrat OpenAPI, schémas Zod ou équivalent.

03 — Modèle de données et règles métier

1. Séparation structurante des concepts

Ne jamais fusionner dans une même table « logement », « annonce » et « réservation ». Le **bien** (`properties`) est une entité immobilière ; l'**unité** (`units`) est une capacité réservable ; l'**offre** (`offers`) décrit la commercialisation (séjour d'abord, puis bail/vente) ; la **réservation** (`bookings`) porte sur l'unité, l'intervalle et le devis figé. Cette séparation prend en charge une villa entière comme une résidence de 12 appartements.

2. Entités à créer en MVP

Table logique	Champs principaux (indicatifs)	Contraintes et responsabilités
`profiles`	`id`, `display_name`, `locale`, `phone_verified_at`	`id` relié à Auth ; minimum de données privées
`organizations`	`id`, `legal_name`, `type`, `status`	Agence, société ou gestionnaire
`memberships`	`user_id`, `org_id`, `role`, `status`	Clé composite ; permissions vérifiées serveur
`owner_relationships`	`property_id`, `owner_profile/org`, `manager_org_id`, `mandate_id`, dates	Distinguer propriétaire et mandataire
`properties`	`id`, `type`, `address_private`, `district_public`, `geo_public`, `review_status`	Adresse privée protégée
`units`	`id`, `property_id`, `name`, `capacity`, `beds`, `status`	Identifiant stable par unité
`unit_amenities`	`unit_id`, `amenity_code`, `value`, `verified_at`	Équipements déclarés/contrôlés
`media_assets`	`id`, `unit_id`, `storage_key`, `kind`, `approved_at`	URL signée si document privé
`offers`	`id`, `unit_id`, `type=stay`, `status`, `ruleset_id`	Réserver `lease/sale` sans activation
`rate_plans`	`id`, `offer_id`, `currency=XOF`, `base_nightly_amount`	Montants entiers en FCFA
`rate_overrides`	`unit_id`, `date`, `amount`, `min_stay`, `rule_priority`	Conflits et antériorité documentés
`calendar_blocks`	`unit_id`, `start_date`, `end_date`, `kind`, `source`	Départ exclusif ; fermetures et imports
`booking_holds`	`id`, `unit_id`, `range`, `expires_at`, `status`	Verrou temporaire, libération explicite
`quotes`	`id`, `unit_id`, `dates`, `line_items_json`, `total_xof`, `expires_at`	Snapshot tarif et politique

Table logique	Champs principaux (indicatifs)	Contraintes et responsabilités
`bookings`	`id`, `quote_id`, `guest_id`, `unit_id`, `dates`, `state`, `policy_snapshot`	Machine à états contrôlée
`payment_intents`	`id`, `booking_id`, `provider`, `provider_ref`, `amount_xof`, `status`	Référence fournisseur unique
`payment_events`	`id`, `intent_id`, `external_event_id`, `verified_payload_digest`, `kind`	Event externe idempotent
`refunds`	`id`, `intent_id`, `reason`, `amount_xof`, `provider_ref`, `status`	Jamais supposé payé avant confirmation
`payouts`	`id`, `organization_id`, `period`, `amount_xof`, `provider_ref`, `state`	Selon capacités PSP contractuelles
`verification_cases`	`id`, `subject_id`, `check_type`, `reviewer_id`, `status`, `valid_until`	Pas de badge global trompeur
`support_cases`	`id`, `booking_id`, `severity`, `status`, `resolution`	Incident et escalade
`audit_events`	`id`, `actor_id`, `resource`, `action`, `before_hash`, `after_hash`, `at`	Historique app append-only
`outbox_events`	`id`, `type`, `aggregate_id`, `payload`, `processed_at`	Notifications et intégrations fiables

Contrainte d'unicité essentielle : (`provider`, `external_event_id`) pour éviter le double traitement d'une notification de paiement. Les imports calendrier doivent aussi posséder une clé stable fournisseur/source.

3. Invariants critiques

1. Dates : `check_in < check_out`, nuitée représentée par intervalle demi-ouvert `[check_in, check_out)` ; deux séjours sont compatibles si l'un sort le jour où l'autre entre.
2. Pour toute unité, deux périodes d'occupation **actives** ne se chevauchent jamais. Dans PostgreSQL, utiliser une contrainte d'exclusion GiST sur `unit_id` et `daterange(check_in,check_out,'[)')`, restreinte aux états réellement bloquants, ou une table d'occupations distincte atomiquement maintenue.
3. Une demande qui n'a pas été acceptée ne bloque pas définitivement le calendrier ; une réservation confirmée requiert la chaîne de paiement réellement vérifiée si paiement requis.
4. Les `holds` expirés ne doivent plus bloquer après transaction de nettoyage fiable ; en cas de retard du job, le contrôle de disponibilité tient compte de `expires_at`.
5. Le total d'un devis est la somme des lignes et taxes configurées ; devise explicitement `XOF` ; valeur non négative ; arrondis définis et testés.
6. Le prix et la politique d'annulation acceptés sont versionnés et figés dans le dossier de réservation ; une modification du tarif public ne modifie pas le passé.
7. Aucune suppression destructive d'une réservation payée, d'un événement PSP, d'un dossier de litige ou d'une écriture de rapprochement.
8. Une organisation ne peut jamais accéder aux données privées d'une autre, y compris via Storage, recherche, exports, liens ou API.

9. L'activation d'une offre dépend du statut de vérification exigé pour la catégorie et de sa date de validité.
10. Toute action sensible (suspension, remboursement, dérogation au tarif) possède acteur, raison, horodatage et trace d'audit.

4. Machines à états

Demande/réservation : `draft → requested → accepted → awaiting_payment → confirmed → checked_in → completed`. États alternatifs : `declined`, `expired`, `payment_failed`, `cancelled_by_guest`, `cancelled_by_host`, `disputed`, `refunded_partially`, `refunded_fully`. Les transitions doivent être définies dans une table de permission, pas par changement de texte libre.

Paiement : `created → pending → succeeded | failed | expired` ; `refunding → refunded | refund_failed`. Le retour navigateur ne peut pas définir `succeeded`.

Annonce : `draft → submitted → under_review → published | changes_requested | rejected` ; puis `suspended` ou `archived`.

Contrôle : `not_started → documents_requested → submitted → under_review → approved | rejected | expired`.

5. Permissions et séparation multi-tenant

- Voyageur : consulter offres publiées, gérer son profil et **ses propres réservations** ; accès limité à l'adresse après confirmation selon contrat.
- Hôte : éditer uniquement les biens sous mandat valable de son organisation ; voir uniquement ses réservations, tarifs et revenus.
- Manager : gérer les tâches autorisées sans pouvoir accéder aux clés, secrets, journaux complets ou remboursements.
- Vérificateur : lire les documents d'un dossier attribué, publier une décision signée, sans supprimer les preuves.
- Finance : rapprochement et remboursements via processus validé, sans publication d'annonce ni accès documentaire non nécessaire.
- Superadmin : privilèges élevés limités, MFA requis et événements journalisés.

La sécurité RLS est une seconde barrière et ne remplace pas l'autorisation métier côté serveur. Interdire l'usage de `service_role` dans le navigateur et les routes publiques.

6. Future-proof sans complexifier la V1

Prévoir les types `stay`, `lease` et `sale` dans le dictionnaire de domaine mais construire uniquement les workflows `stay`. Ne pas créer prématurément des tables de crédit, séquestre, publicité, signatures ou estimation IA ; les extensions devront s'appuyer sur `property_id`, `unit_id`, `offer_id`, `organization_id`, `subject_id` et l'historique des documents. Utiliser des migrations versionnées et des clés étrangères réelles.

7. Migration initiale et jeux de données

Lot initial : profils/organisations → biens/unités → tarifs/calendriers → devis/réservations → paiements → conformité/audit. `seed.sql` : 2 organisations fictives, 3 propriétaires de test, 6 unités, saisons tarifaires variées, chevauchements et cas de remboursement synthétiques. Ne jamais mettre d'identités personnelles réelles ni de secrets dans les seeds.

Critère d'acceptation données : tests SQL empêchant 100 % des chevauchements actifs simulés, tests RLS inter-organisations et restauration d'une sauvegarde en environnement isolé.

04 — API, réservation, paiement et synchronisation

1. Contrats API MVP (proposition)

Méthode / endpoint	Rôle	Corps / retour	Sécurité
`GET /api/v1/search`	Voyageur	destination, dates, invités, filtres → unités disponibles présumées	Lecture publique limitée, rate limit
`GET /api/v1/units/:id`	Voyageur	fiche, médias, prix de départ, règles	Publication requise
`POST /api/v1/quotes`	Voyageur	unité, dates, invités → total détaillé + expiration	Revalidation côté serveur
`POST /api/v1/booking-requests`	Voyageur	quote_id, identifiant idempotent	Compte + contrôle anti-abus
`POST /api/v1/bookings/:id/accept`	Hôte	acceptation conditionnelle	Organisation mandatée
`POST /api/v1/bookings/:id/decline`	Hôte	refus et motif	Organisation mandatée
`POST /api/v1/bookings/:id/payment-intent`	Voyageur	moyen de paiement → URL/ intent PSP	Quote figé, rôle, état
`POST /api/v1/webhooks/payments/:provider`	PSP	notification brute signée	Signature, timestamp, anti-replay
`POST /api/v1/bookings/:id/cancellation`	Voyageur/hôte	motif, restitution estimée	Politique snapshot
`GET /api/v1/host/reservations`	Hôte	liste paginée, statuts	RLS + org_id
`POST /api/v1/host/units`	Hôte	données unité	Vérification mandat/ organisation
`PATCH /api/v1/host/calendar`	Hôte	plage, tarifs, blocages	Versionnement et contrôle conflits
`POST /api/v1/control/reviews/:id/decision`	Réviseur	décision et motifs	Accès privilégié et audit
`GET /api/v1/health`	Ops	statut de service sans secrets	Protection contre divulgation

Tous les endpoints mutateurs valident le payload (`Zod` ou équivalent), utilisent des autorisations explicites et génèrent un `request_id`. Versionner les contrats (`/api/v1`), documenter les erreurs métier 400/401/403/409/422/429/503.

2. Scénario de réservation recommandé

1. Rechercher des unités selon dates. La page de recherche est informative : la disponibilité n'est **pas un engagement** tant que le serveur n'a pas posé un blocage transactionnel.
2. Calculer un devis serveur avec lignes explicites : nuits, nettoyage éventuel, frais ChezVous, taxes applicables, remise ; stocker version de règle et échéance.
3. Voyageur crée une **demande** et reçoit un accusé de réception avec échéance d'acceptation. Aucun débit automatique à ce stade par défaut.
4. Hôte accepte ; le service atomique revérifie disponibilité et crée une réservation/occupation contrôlée ou un `hold` temporaire avec expiration.
5. Paiement via PSP choisi ; service crée un `payment_intent` associé au devis et à la réservation. Retour navigateur « merci » non autoritaire.
6. Webhook PSP : vérifier signature, référence, montant, devise, événement unique, statut fournisseur ; optionnellement relire le statut depuis l'API PSP. Enregistrer l'événement, faire la transition et publier `booking.confirmed` dans `outbox` **dans la même transaction**.
7. Notifications après commit avec tentatives contrôlées et non bloquantes ; hôte reçoit les données d'arrivée autorisées.
8. Si délai dépassé, annulation/expiration via tâche planifiée transactionnelle ; distinguer paiement tardif (quarantaine et processus restitution) d'une réussite valide.

La réservation instantanée sera activée uniquement pour des unités dont les calendriers sont fiabilisés, les règles d'annulation paramétrées et les procédures d'arrivée vérifiées.

3. Algorithme anti-double réservation

Pseudo-SQL conceptuel (à adapter aux états retenus dans les migrations) :

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
ALTER TABLE unit_occupancies
  ADD CONSTRAINT no_active_overlap
  EXCLUDE USING gist (
    unit_id WITH =,
    daterange(check_in, check_out, '[)') WITH &&
  ) WHERE (status IN ('held','confirmed','checked_in'));
```

Important : maintenir les expirations de `held` par transition SQL atomique, et ne pas employer une condition basée directement sur `now()` dans un index partiel. Le statut doit refléter l'expiration via le job et le service métier. Les conflits d'exclusion doivent devenir une erreur API `409 UNAVAILABLE`, et la tentative de réservation doit être rejouable sans paiement dupliqué.

4. Adaptateur PaymentGateway

Contrat interne minimal :

```
interface PaymentGateway {
  createPayment(input: PaymentRequest): Promise<ProviderIntent>;
  verifyWebhook(rawBody: Uint8Array, headers: Headers): Promise<VerifiedEvent>;
  retrievePayment(providerRef: string): Promise<PaymentStatus>;
  requestRefund?(input: RefundRequest): Promise<RefundResult>;
```

```
  requestPayout?(input: PayoutRequest): Promise<PayoutResult>;
}
```

Sélection du prestataire après réponses écrites sur : statut réglementaire, marques/moyens supportés au Sénégal, tarif par transaction, frais fixes, annulations, disponibilité API, remboursement total/partiel, reverses, temps de règlement, chargebacks, identité hôte et facturation. Une API de paiement **n'implique pas** un service de séquestre ou de split-payout réglementaire.

5. Rapprochement et comptabilité

- Référence interne de chaque opération + référence unique PSP ; balance journalière par devise.
- Tableau distinct : montant voyageur, montant propriétaire, commission ChezVous, frais PSP, taxe applicable, remboursement, ajustement, coût de service.
- Import ou interrogation des mouvements PSP ; différences consignées et résolues par utilisateur autorisé.
- Les revenus reconnus par ChezVous dépendent du contrat et de la comptabilité, pas simplement du volume brut des réservations.
- Conserver les pièces justificatives et appliquer les durées de conservation légales validées.

6. Annulation, no-show et litige

Politiques explicites `flexible`, `moderate`, `strict` personnalisables dans des bornes conformes, chacune versionnée. Tableau calculé du montant remboursable selon heure locale et date d'arrivée, jamais modifiable a posteriori. Prévoir `host_cancelled` avec assistance au relogement éventuel ; désaccord sur état du bien ouvre un dossier horodaté et documenté ; remboursements uniquement par actions fournisseur authentifiées.

7. Synchronisation Airbnb/Booking et autres canaux

- MVP : date bloquée manuellement + calendrier interne ; hôte choisit mode « confirmation requise ».
- V1.5 : abonnements iCal import/export à fréquence documentée ; protection SSRF, téléchargements en taille limitée, validation du format et de la provenance, détection d'échecs.
- iCal n'est pas instantané ; les séjours provenant d'un autre canal peuvent créer une course. Ne jamais promettre « zéro double réservation » sur la seule base d'un import iCal.
- API partenaires Airbnb/Booking.com seulement après autorisation contractuelle ; ne pas contourner leurs restrictions via scraping ou cookies non autorisés.

8. Notifications

Envoi d'un e-mail transactionnel pour chaque étape clé, puis WhatsApp officiel seulement avec base juridique/consentement et modèles approuvés. Les notifications n'ont pas le droit de changer un état métier ; une notification échouée est rejouée depuis `outbox`, tandis que la réservation demeure vraie en base.

9. Critères d'acceptation API

`409` systématique pour conflit dates ; appels répétitifs avec même clé idempotente ne créent qu'une seule opération ; fausse signature PSP rejetée ; événement PSP répété ignoré sans second débit ; montant ou devise différent mis en quarantaine ; remboursement partiel correctement tracé ; une panne mail ne défait pas une réservation confirmée ; recherche rapide sans fuite de données privées.

05 — Sécurité, confidentialité, conformité et contrôle des risques

1. Contexte sénégalais : vérifier avant toute mise en production

- **Hébergements touristiques** : le ministère sénégalais compétent publie les procédures d'agrément liées au décret n° 2005-145 du 2 mars 2005. Il faut confirmer précisément quels appartements meublés, résidences, maisons et exploitants relèvent de chaque exigence, ainsi que les pièces et dates d'expiration. Source : https://mcat.gouv.sn/service/demande-dagrement-pour-les-etablissements-dhebergement-touristique/ . La plateforme n'accorde pas elle-même d'agrément.
- **Paiements** : instruction BCEAO n°001-01-2024 du 23 janvier 2024 relative aux services de paiement dans l'UMOA. Source : https://www.bceao.int/fr/reglementations/instruction-ndeg001-01-2024-du-23-janvier-2024-relative-aux-services-de-paiement . Contrat avec prestataire habilité avant encaissement, reversement ou conservation éventuelle de fonds pour tiers.
- **Données personnelles** : loi sénégalaise n°2008-12 du 25 janvier 2008, autorité CDP, formalités et transferts selon les cas. Source : https://www.archives.sn/docs/codes/loi-protection-donnees-a-caractere-personnel . Une analyse juridique des traitements et de l'hébergement transfrontalier est nécessaire.
- **Location résidentielle/vente futures** : documents, mandats, règles de dépôt et frais, profession immobilière, règles foncières et notariat à examiner séparément avant activation de ces modules.

Cette section est une checklist de conception, **pas un avis juridique ni une garantie de conformité**. Désigner un conseil juridique au Sénégal et un responsable opérationnel des dossiers avant le pilote payant.

2. Modèle de menaces prioritaire

Menace	Impact	Contrôle recommandé	Test obligatoire
IDOR : client lit la réservation d'autrui	Fuite de données privées	Autorisation métier + RLS par utilisateur/org	requêtes inter-comptes → 403
Double réservation simultanée	Rejet client, remboursement	Contrainte DB exclusion + transactions	100 requêtes concurrentes
Webhook PSP falsifié/rejoué	Faux paiement, double crédit	Signature, idempotence, relecture PSP	signatures/fausses devises/ replays
Escalade de privilèges d'agent	Fraude annonces et remboursements	RBAC, MFA, séparation des fonctions	tests par rôle
Injection SQL / XSS	Vol données et sessions	Requêtes paramétrées, validation, CSP	scanners et tests de charge
Téléchargement justificatifs arbitraires	Documents privés exposés	Buckets privés, URL signées, analyse fichiers	essais d'accès anonymes

Menace	Impact	Contrôle recommandé	Test obligatoire
Faux logement / faux mandat	Préjudice financier	KYC/mandat/révision, conservation preuves	échantillonnage audits
SSRF via import iCal ou images	Attaque services internes	URLs autorisées, réseau sortant limité	URL locale et redirect chain
Vol de clés d'intégration	Accès aux paiements/ données	Secrets hors Git, rotation, permission minimum	secret scanning + runbook
Suppression malveillante	Perte d'inventaire/argent	Historique/audit, sauvegardes, double validation	restauration testée
Abus marketing	Coût SMS/WhatsApp excessif	Quotas, opt-in, rate limiting	tests abus

3. Politique de sécurité applicative

- **Auth** : Supabase Auth SSR ; cookies session correctement sécurisés ; MFA pour finance/admin ; protection des flux de réinitialisation ; aucune session dans localStorage si architecture SSR possible.
- **RBAC / ABAC** : utilisateur + organisation + mandat + statut du bien + fonction ; vérifier permissions sur chaque action sensible, et non uniquement l'affichage d'un bouton.
- **RLS** : lecture publique exclusivement sur vues d'annonces publiées et champs non sensibles ; autres tables en politique par org et utilisateur. `service_role` exclusivement sur serveur protégé et routes internes.
- **Documents** : séparation photos publiques / justificatifs privés ; validation MIME réelle, taille maximale, conversion images, scan antivirus si disponible, URL temporaires et journal accès.
- **Application** : protection CSRF adaptée aux flux, CSP, headers de sécurité, sanitation de contenu saisi, limitation de débit, contrôle upload, logs sans PII ni tokens.
- **Infrastructure** : production/staging/projet test séparés, politiques de branches, protection `main`, permissions des équipes minimales, secrets via gestionnaire, backups automatiques et restauration pratiquée.
- **Supply chain** : mise à jour dépendances, Dependabot/Renovate après revue, `npm audit` ou équivalent, SCA, SAST, secret scanning, SBOM et inventaire des licences.

4. Processus de confiance ChezVous Trust

Niveaux de contrôle distincts : (1) téléphone/e-mail vérifié, (2) identité du professionnel vérifiée, (3) mandat/qualité d'exploitant contrôlé, (4) logement effectivement visité/documenté, (5) autorisations d'exploitation contrôlées et à jour. Chaque badge doit avoir portée, date, vérificateur et limitations. L'utilisateur ne doit jamais lire « propriété garantie » si seul un numéro de téléphone est confirmé.

Fiche de contrôle terrain : adresse exacte comparée au mandat ; existence et état des pièces ; détecteurs/ sécurité selon obligations locales ; compteur équipements ; cohérence photos ; Wi-Fi si annoncé ; procédure de clés ; capacité réelle ; propreté ; coordonnées urgences ; date de contrôle.

5. Protection des données et cycle de vie

Inventaire des traitements et bases légales, mentions d'information, minimisation, durée de conservation selon base juridique, procédure d'accès/correction/suppression, transfert hors Sénégal, sous-traitants et journal consentements. Les rapports agrégés ne doivent pas réidentifier les voyageurs. Définir stratégie de sauvegarde et d'effacement compatible avec les obligations comptables et les litiges.

6. Gestion d'incident et continuité

- Gravité **P0** : faux paiement, fuite de données, corruption réservations, double occupation confirmée ; incident déclaré immédiatement, gel sélectif des flux concernés.
- **P1** : indisponibilité réservation/paiement, notifications critiques échouées ; fonctionnement dégradé et file de traitement manuelle.
- **P2** : lenteurs et erreurs UX non critiques.
- Runbooks : paiement bloqué, double booking externe, hébergement inaccessible, remboursement en attente, compte compromis, panne du PSP, restauration DB.
- Objectifs **à contractualiser** et tester : RPO et RTO selon capacité des sauvegardes et budget ; ne pas annoncer une disponibilité de 99,99 % sans instrumentation.

7. Portes de lancement (no-go si non satisfaites)

1. Accord juridique sur hébergements, contrats, politique d'annulation, responsabilité de ChezVous.
2. Paiement PSP contractualisé, webhooks validés, remboursement et rapprochement testés.
3. Test de concurrence anti-double réservation réussi.
4. Sécurité RLS vérifiée par tests automatisés et revue humaine.
5. Procédure support hôte/voyageur disponible pendant les séjours pilotes.
6. Disponibilité/conditions de chaque unité pilote contrôlées.
7. Sauvegarde/restauration et journal d'audit démontrés.

06 — Roadmap, lots de développement et critères de validation

1. Cadence de travail

Proposition indicative : **12 à 16 semaines** pour un MVP pilote, sous réserve d'une équipe disponible, des contrats prestataires et de l'accessibilité des premiers logements. Cette durée n'est pas un engagement et devra être recalculée avec l'équipe. Travail en sprints de 2 semaines, démonstration fonctionnelle et rétro chaque sprint. Publier un journal de risques, un journal de décisions d'architecture et l'avancement vérifiable par issue/PR/test.

2. Préparation (G0) : cadrage et décision — 1 à 2 semaines

- Valider stratégie de mise en marché : 20-30 unités professionnelles à Dakar, demande de réservation prioritaire, commission 7 % **à tester**, abonnements optionnels après pilote.
- Obtenir fichiers de logo/charte officiels ChezVous, figer tokens design light, contrastes et composants.
- Valider les catégories d'hébergement, obligations et preuves requises avec conseil local ; lancer consultation PSP (PayDunya, PayTech et autres acteurs habilités).
- Désigner product owner, référent technique, référent opérations/support, juriste/comptable et responsable de la gestion des accès.
- Finaliser diagrammes, contrat des états métier et critères d'acceptation de chaque module.

Go/no-go : exigences classées MUST/SHOULD/LATER, politique PSP réalisable, permissions approuvées, registre risques établi.

3. Lot 1 — Infrastructure et identité — 2 semaines indicatives

Stories : initialiser dépôt privé GitHub ; règles branches et PR ; installer Next.js TypeScript, Tailwind, shadcn/ui ; CI lint/typecheck/tests/build ; environnements staging/prod séparés ; Supabase migrations ; Auth SSR ; organisations, rôles, invitations ; design system accessible ; journaux sans secrets.

Critères : inscription connexion déconnexion et réinitialisation ; test d'isolation entre deux organisations ; déploiement Preview depuis PR ; migrations rejouables sur base neuve ; score performance initial relevé.

Dépendances : G0.

4. Lot 2 — Catalogue et vérification — 2 semaines

Stories : propriétés, unités et offres stay ; import photos ; champs normalisés ; équipements ; quartiers ; fichier documents privés ; soumission, revue et publication ; vitrine publique SEO ; recherche simple, filtres et fiche logement ; visite test du workflow admin.

Critères : unité non vérifiée jamais visible publiquement ; propriétaire ne modifie pas unité d'une autre organisation ; photos publiques optimisées ; pièces privées inaccessibles ; application utilisable sur téléphone.

Dépendances : lot 1.

5. Lot 3 — Disponibilité, tarifs et réservation — 3 semaines

Stories : modèle tarif/nuit, calendriers, exceptions, blocs manuels, devis et taxes configurables, politique d'annulation versionnée, demande de réservation, acceptation hôte, occupation bloquante atomique, expiration des demandes et holds, notifications de statut.

Critères : aucune collision sur 100 requêtes simultanées dans les tests ; devis total reproductible ; dates d'entrée/sortie cohérentes ; `409` en conflit ; hôte ne confirme pas un bien sans mandat.

Dépendances : lots 1, 2.

6. Lot 4 — Paiement et exploitation — 2 à 3 semaines

Stories : adaptateur PSP sandbox ; paiement, webhooks signés, idempotence, confirmation, échec, expiration, remboursements si API autorisée ; tableau de rapprochement ; dashboard host/control ; e-mails ; gestion d'incidents et journal d'audit.

Critères : faux webhooks rejetés ; événement répété traité une fois ; différence montant XOF bloquée ; aucune confirmation sur retour navigateur seul ; remboursement d'essai traçable ; reprise après échec e-mail.

Dépendances : contrat PSP et lot 3.

7. Lot 5 — Qualité, sécurité et pilote — 2 à 3 semaines

Stories : parcours mobile et accessibilité, E2E voyageur/hôte/admin, tests charge, audits RLS et sécurité, backup/restauration, analytics, formation support, onboarding 20-30 logements, test de 3 à 5 réservations réelles autorisées et pilotes.

Critères : gates conformité et paiements satisfaites, personnel support identifié, cas de crise testés, taux de réservations honorées suivi, décision documentée pour ouverture publique.

8. Après MVP : découpage progressif

Phase	Conditions de déclenchement	Fonctions
V1.1	Fiabilité du séjour validée	Avis séjour réel, favoris, meilleure recherche, mini-page propriétaire
V1.5	Propriétaires actifs, demandes B2B prouvées	Abonnements, liens directs, export/import iCal, statistiques, équipe
V2	3+ mois de données qualitatives	Prix saisonniers avancés, intégrations canaux autorisées, CRM léger, abonnements agences

Phase	Conditions de déclenchement	Fonctions
V3	Problèmes juridiques/ops résolus	Location longue durée (visites, dossiers, bail, loyers)
V4	Réseau vérifié et partenariats établis	Vente, programmes neufs, courtage et dossiers notariaux
V5	Données suffisantes et coût maîtrisé	Assistant IA, suggestion de prix, analyse comparative, langues supplémentaires

La séquence future est une priorité relative, **pas un calendrier garanti**. Ne pas ouvrir la vente et la location si cela déstabilise l'expérience de réservation.

9. Responsabilités / RACI indicatif

Activité	Product owner	Lead dev	Dev frontend	Dev backend	QA/Ops	Juridique/ Finance
Périmètre et priorités	A	C	C	C	C	C
Architecture et schéma	C	A	C	R	C	C
UI/UX et accessibilité	A	C	R	C	R	-
Réservation et paiements	C	A	R	R	R	C
Sécurité et exploitation	C	A	R	R	R	C
Validations réglementaires	C	C	-	C	R	A
Pilote hôtes et service	A	C	-	-	R	C

`A` : responsable final, `R` : exécution, `C` : consulté. Plusieurs activités peuvent être réunies chez la même personne dans une petite équipe, mais les revues de paiement/sécurité doivent conserver un second regard.

10. Enveloppe budgétaire indicative (hypothèses, pas devis)

La variabilité salariale, le recours aux indépendants, le contenu photo et les coûts PSP empêchent d'annoncer un montant fiable sans devis. Construire trois scénarios :

- **Équipe fondatrice / lean** : 1 développeur full-stack + 1 designer ponctuel + QA externalisée + opérations manuelles ; calendrier plus long, risque personne clé.
- **Petite équipe dédiée** : lead/full-stack, frontend, QA part-time, product/ops et design ponctuel ; compromis délai/qualité.
- **Agence externe** : coût contractualisé par lots et critères d'acceptation ; exige propriété du dépôt/ code/données et transfert de compétences.

Postes de budget mensuels à chiffrer : développement, hébergement, DB, stockage, trafic photo, email/WhatsApp, cartographie, paiement, assistance, contrôles terrain, juridique, marketing, réserves incidents. Budget de lancement distinct des coûts de fonctionnement ; inclure 15-25 % de réserve d'imprévus **comme hypothèse**.

11. GitHub Projects : mécanique de pilotage

Epics `EPIC-01` à `EPIC-09` et issues détaillées dans `backlog_chezvous.csv`. Colonnes : Backlog → Ready → In progress → Review → QA → Done, et Blocked (motif). Chaque issue possède : objectif, dépendance, tests, niveau de priorité, preuve et lien PR ; % d'avancement calculé par tâches réellement terminées, non par impressions subjectives.

Definition of Ready : story comprise, design ou contrat disponible, données/permissions définies, critères écrits, dépendances confirmées.

Definition of Done : code relu, tests passants, scan sans risque bloquant, docs/changelog mis à jour, capture ou preuve de démonstration, déployé staging, acceptance product, statut GitHub à jour.

12. Critères de Go/No-Go vers le marché

Inventaire contrôlé et juridiquement autorisé, réponse opérationnelle en temps utile, support des incidents, mesure des annulations, commission nettement supérieure aux frais variables ou stratégie explicite de subvention pilote, taux d'échec paiement connu, tests de sécurité et restauration réussis. Le lancement ne dépend pas d'un simple total de fonctionnalités terminées.

07 — Politique GitHub et références open source

Date de vérification documentaire : 18 octobre 2026. Les dépôts ci-dessous ont été retrouvés via GitHub ; licences principales examinées dans les fichiers du dépôt et/ou documentation officielle. L'examen n'est **pas** un audit complet de sécurité de leurs versions transitives. Avant intégration, épingler une version/commit précis, vérifier vulnérabilités et tester les chemins critiques.

1. Candidats à intégrer après validation

Repo	Usage proposé	Licence principale	Évaluation / limites	Décision
https://github.com/shadcn-ui/ui	UI premium, composants React	MIT	dépôt actif/non archivé, composants copiés et adaptés à notre charte	**Adopter**
https://github.com/gpbl/react-day-picker	calendrier date arrivée/départ	MIT	librairie d'interface, ne fournit pas anti-overbooking	**Adopter**
https://github.com/fullcalendar/fullcalendar	calendrier Host	MIT **standard seulement**	Premium : contrat commercial/AGPL distinct, ne pas l'utiliser sans autorisation	**Évaluer**
https://github.com/maplibre/maplibre-gl-js	carte interactive et affichage biens	BSD-3-Clause	fournisseur de tuiles/géocodage à budgéter séparément	**Adopter**
https://github.com/TanStack/query	requêtes/caches côté client	MIT	usage sélectif ; ne jamais prendre son cache pour vérité transactionnelle	**Adopter selon besoin**
https://github.com/react-hook-form/react-hook-form	formulaires d'onboarding	MIT	combiner validation serveur, ne pas s'arrêter au client	**Adopter**
https://github.com/colinhacks/zod	validation des contrats TypeScript	MIT	schémas API et formulaires ; contrôle de droits ailleurs	**Adopter**
https://github.com/supabase/ssr	cookies/auth Next.js SSR	MIT	projet non archivé ; éviter les anciens auth helpers dépréciés	**Adopter**
https://github.com/refinedev/refine	dashboard admin/CRUD	MIT	nécessite prototype pour vérifier coût d'intégration au design ChezVous	**Prototyper**
https://github.com/marmelab/react-admin	alternative dashboard admin	MIT pour le coeur	options Enterprise et design MUI ; choisir un seul framework admin	**Comparer / non cumulatif**

2. Références fonctionnelles et non sources à copier telles quelles

Repo	Pourquoi l'étudier	Limite majeure	Décision
https://github.com/Qloapps/QloApps	PMS hôtelier complet, modèle réservation, opérations	PHP/MySQL, noyau OSL-3.0 et modules sous autres licences ; différentes obligations	**Inspiration fonctionnelle**, pas intégration directe
https://github.com/calcom/cal.com	gestion avancée des disponibilités et prise de RDV	licences copyleft/ commerciales selon versions/ composants ; inadéquation métier hébergement	**Étude de concepts uniquement**, pas copie
https://github.com/vercel/next.js	frameworks et exemples techniques	dépôt du framework, pas plateforme immobilière	**Base technologique**, pas application prête
https://github.com/supabase/supabase	infrastructure de référence et pratiques de sécurité	plateforme complète, modules/services variés	**Documentation + outils**, pas fork complet

Pourquoi ne pas prendre un clone Airbnb populaire ? Beaucoup de clones démontrent des écrans sans gestion sérieuse des réservations concurrentes, reversements, licences de contenu, sécurité multi-tenant et conformité locale. La sélection se fait sur les invariants métiers, pas sur les étoiles GitHub.

3. Scorecard d'adoption obligatoire

Attribuer une note de 0 à 5 à chaque critère, avant merge :

1. **Licence et obligations** : autorisation usage commercial/SaaS, attribution, redistribution, copyleft, assets.
2. **Activité** : date de dernière release/commit, issues, correctifs de sécurité, équipe mainteneurs.
3. **Sécurité** : dépendances connues, advisory, CVE, politique de divulgation, capacité de patch.
4. **Compatibilité** : React/Next.js/Node/Supabase, SSR, navigateur mobile, taille bundle.
5. **Adéquation fonctionnelle** : quel besoin résout-il ? complexité à supprimer si non utilisé.
6. **Performance et accessibilité** : tests mobiles, clavier, WCAG, métriques.
7. **Maintenance future** : lock-in, risques de rupture, test de mise à jour.

Règle d'arrêt : licence incompatible, absence d'autorisation claire de réutiliser assets ou code, vulnérabilité critique non remédiée dans un flux sensible, ou incompatibilité irréductible.

4. Cadence permanente de recherche et mise à jour GitHub

- **À chaque nouvelle fonctionnalité** : rechercher au moins 3 dépôts ou bibliothèques, comparer et rédiger un `ADR`.
- **Une fois par mois pendant le développement** : revoir dépendances, CVE, maintenance, mises à jour et dépôts alternatifs ; mettre à jour ce catalogue.
- **Avant une release majeure** : SBOM et examen des licences, audit supply-chain, suppression des dépendances inutiles.
- **Après un incident** : décider si l'outil tiers doit être patché, remplacé ou isolé.

Le catalogue de références appartient exclusivement au projet **CHEZVOUS**, distinct des projets ODACE HOME et autres verticales.

5. Ordre de comparaison pour les premiers composants

1. UI : shadcn/ui vs composants développés maison, décision shadcn/ui.
2. Calendrier d'arrivée/départ : DayPicker.
3. Vue Host : FullCalendar Standard, puis comparer au besoin un calendrier développé sur mesure pour la grille d'unités (Premium payant).
4. Admin : Refine vs React-admin vs composants internes, **une seule** solution retenue après prototype.
5. Paiement : SDK/officiels PSP et documentation validée, aucun dépôt tiers inconnu pour la gestion de fonds.
6. Moteur réservation : **code métier propriétaire**, contrôles SQL et tests ; s'inspirer de QloApps sur le processus, pas sur le code.

6. Références techniques officielles

- Supabase sécurité RLS : https://supabase.com/docs/guides/database/postgres/row-level-security
- Supabase SSR : https://github.com/supabase/ssr
- FullCalendar licence : https://fullcalendar.io/license
- PayDunya API déboursement : https://developers.paydunya.com/doc/FR/api_deboursement
- BCEAO instruction paiement : https://ns2.bceao.int/fr/reglementations/instruction-ndeg001-01-2024-du-23-janvier-2024-relative-aux-services-de-paiement
- QloApps licence et stack : https://github.com/Qloapps/QloApps

7. Traçabilité de réutilisation

Ajouter dans chaque PR important : `Dépôt`, `version/commit`, `licence`, `notice conservée`, `tests`, `vulnérabilités`, `pourquoi retenu`, `alternative rejetée`. Les assets images, logos et données d'un projet open source ne sont pas automatiquement réutilisables sous la même licence que son code.

08 — Test, observabilité, exploitation

1. Pyramide de test

- **Unitaires (Vitest ou équivalent)** : calculs XOF, périodes [arrivée, départ), arrondi, durée minimale, tarifs saisonniers, commission par provenance, règles d'annulation.
- **Intégration DB** : SQL transactionnel, GiST anti-chevauchement, holds expirés, RLS multi-tenant, storage privé.
- **Contract tests PSP** : signatures, doubles webhooks, inversions d'ordre, frais, refus, succès après expiration, remboursement partiel et total.
- **E2E (Playwright)** : recherche à réservation et parcours hôte/admin sur mobile et desktop.
- **Non-fonctionnel** : accessibilité WCAG, SEO, performance mobile, charge, pertes de réseau, sauvegardes/restauration, sécurité API OWASP.

2. Matrice de tests d'acceptation critiques

ID	Scénario	Résultat indispensable
T01	2 demandes simultanées et une seule unité	une seule plage active confirmable
T02	100 checkouts concurrents sur même unité/dates	pas de double booking actif
T03	date sortie A = entrée B	réservation contiguë autorisée
T04	changement prix entre devis et paiement	devis figé honoré jusqu'expiration ou invalidé clairement
T05	client modifie montant HTTP	rejet et recalcul serveur
T06	3 callbacks PSP `success` identiques	1 seul paiement métier, 1 seule confirmation
T07	webhook `success` tardif après expiration	pas de confirmation illégitime, dossier finance
T08	webhook signé incorrectement	aucun changement de statut
T09	voyageur A accède booking de B	403/404 sans fuite
T10	hôte agence A consulte agence B	aucune ligne retournée, y compris storage
T11	annulation éligible remboursement	montant exact, suivi PSP, message clair
T12	frais plateforme appliqués à host-direct	pas de commission distribution host-direct
T13	calendrier iCal indisponible	alerte et mode demande, pas réservation instantanée risquée
T14	unité non vérifiée	invisible dans résultats publics

ID	Scénario	Résultat indispensable
T15	coupure réseau pendant paiement	état retrouvable, pas de double charge créée
T16	contrôle de droits admin	refus sans MFA/privilège, événement audit
T17	restauration sauvegarde récente	service reconstitué sans incohérences majeures
T18	téléphone 360px, réseau lent	recherche et réservation utilisables

3. CI/CD et GitHub

Chaque PR déclenche format, lint, type-check, unit tests, analyse de dépendances et vérification secret. Branche `main` protégée et revue obligatoire. Tests d'intégration sous DB de test ; secrets sandbox exclusivement en coffre CI. Release candidate en staging ; migration SQL vérifiée ; test manuel GO/NO-GO avant production.

4. Observabilité

Tableaux de bord : latence page/API, erreurs 5xx, échecs de connexion, tentatives webhooks non traitées, demandes expirées, erreurs iCal, paiements orphelins, différence PSP vs interne, incidents par unité. Seuils P0/P1 avec escalade et responsabilité identifiée.

5. Objectifs SLO pilotes (à valider)

- disponibilité du parcours réservation cible : 99,5 % par mois pendant pilote ; mesure côté utilisateur.
- anomalie paiement non rapprochée de plus de 24 h : aucune en régime normal ; alerte.
- échecs de synchronisation iCal : visibles et traitables.
- RPO/RTO indicatifs : 24 h / 4 h **à revoir selon capacité technique et exigences du PSP** ; pour paiements les journaux externes et événements doivent permettre une reconstitution exhaustive.
- première réponse support durant horaires définis : médiane cible < 10 min ; hors horaires, indiquer le délai.

6. Runbook incident

1. Identifier l'incident, prioriser P0–P3, activer on-call.
2. Pour paiements/réservations : si besoin passer temporairement au mode demandes sans paiement ou désactiver réservation immédiate.
3. Sauvegarder journaux/identifiants, corriger sans altérer les historiques financiers.
4. Communiquer avec les réservations affectées, initier remboursements par PSP si nécessaires.
5. Vérifier les journaux et la réconciliation, puis produire une analyse de cause racine.

7. Qualité d'exploitation

Une plateforme d'hébergement a une composante physique. Chaque hôte doit renseigner un contact joignable, une procédure d'accès au logement, une règle de remise des clés, une politique de dépannage et un canal d'urgence. Le support ChezVous ne doit pas promettre une conciergerie que son organisation ne peut pas réaliser.

09 — Modèle économique, hypothèses et indicateurs

1. Modèle recommandé

Marketplace à **commission de distribution** sur les séjours dont ChezVous est apporteur d'affaires ; logiciel de gestion avec **abonnement mensuel** pour les hôtes qui utilisent une mini-page de réservation directe ; services locaux optionnels facturés séparément (photos, inspection, ménage, maintenance, via partenaires contractuels).

2. Hypothèses commerciales à tester, non tarifs définitifs

Offre	Proposition	Condition
Free	publication de base gratuite ; 7 % sur réservations marketplace	un nombre limité d'unités, conformité obligatoire
Pro	abonnement env. 19 900 FCFA/mois	outils de gestion et réservation directe sans commission de distribution
Agency	abonnement env. 49 900 FCFA/mois	plusieurs unités et collaborateurs
Services	facture à la prestation	marge/prix clairement expliqués, sous-traitance contractuelle

Les paiements, remboursements, reversements et impôts ne se confondent pas avec la commission. Les abonnements Pro/Agency couvrent le logiciel et non les services de terrain illimités.

3. Exemple unitaire simple

Hypothèse : séjour de 150 000 FCFA ; commission marketplace 7 % = 10 500 FCFA. Frais de paiement 2 % du séjour = 3 000 ; support 1 000 ; réserve incident 1 000 ; acquisition 2 500. Contribution = **3 000 FCFA**, hors charges fixes, taxes, éventuels coûts d'authentification et remboursements réels.

À 6 % de commission avec les mêmes coûts, la contribution tombe à 1 500 FCFA. Si acquisition ou paiement sont plus chers, elle peut devenir négative.

4. Scénarios de volume mensuel (simulation seulement)

Volume mensuel	GMV avec 150 000 FCFA / séjour	CA commission à 7 %	Contribution à 3 000 FCFA / séjour
20 réservations	3 000 000	210 000	60 000
50 réservations	7 500 000	525 000	150 000
100 réservations	15 000 000	1 050 000	300 000

Volume mensuel	GMV avec 150 000 FCFA / séjour	CA commission à 7 %	Contribution à 3 000 FCFA / séjour
200 réservations	30 000 000	2 100 000	600 000

Ce tableau n'est **pas** un bénéfice net. Ajouter les revenus des abonnements, puis retrancher leurs coûts de service et les charges fixes. Aucun scénario ne garantit qu'un inventaire de 20–30 unités puisse générer ces réservations : mesurer occupation et canal d'acquisition.

5. KPI et instrumentations

- **Unités actives fiables** : unités publiées ayant toutes les pièces applicables, calendrier validé et contact joignable.
- **Search-to-book** : réservations confirmées / sessions de recherche qualifiées.
- **Quote-to-pay** : paiements confirmés / devis soumis, par canal.
- **Host response time** : médiane délai réponse ; délais expirés.
- **Booking honored** : check-in effectué / réservations confirmées hors cas clairement catégorisés.
- **Double booking** : 0 tolérance dans le référentiel interne.
- **Contribution par séjour** : commission moins coûts variables attribuables.
- **Direct booking mix** : part réservations provenant de liens hôtes ; distincte de marketplace.
- **Host activation** : délai inscription → 1ère unité contrôlée → 1ère réservation.
- **Retention** : hôtes actifs au mois 1, 3, 6 ; voyages répétés, cohortes sans faux chiffres.
- **Chargebacks et remboursements** : montant et cause, non confondus avec annulation.

6. Go-to-market

Approvisionnement d'abord : contacter gestionnaires de 5–15 unités et propriétaires réactifs ; sélectionner 20 à 30 unités à Dakar. Contrôler photos, calendrier, conditions de remise de clés, modalités de contact et preuves de conformité. Activer recherche SEO locale, liens WhatsApp traçables des hôtes, partenariats, contenu de confiance et recommandations voyageurs. Tester plusieurs messages commerciaux avant campagnes payantes massives.

7. Épreuves de prix

A/B ou enquête prix auprès d'hôtes : 6 %, 7 %, 8 % marketplace selon valeur effectivement délivrée ; Pro entre 19 900 et 29 900 FCFA / mois ; distinguer la tarification des autres concurrents. Mesurer intention de payer et achats réels, pas seulement avis déclaratifs. Séparer droits/frais de conformité et commission.

8. Décisions de marché à ne pas inventer

Les parts de marché, coûts d'acquisition, taux d'occupation par ville, revenus des concurrents locaux et niveaux de réservations réelles n'ont pas été audités. Les benchmarks des plateformes sénégalaises précédemment documentés sont un point de départ, pas une preuve de demande. Le modèle doit être révisé avec données partenaires et tests payants.